

AUTOMI E MACCHINE - DEFINIZIONI ED EQUIVALENZE

Un **Automa Finito Deterministico**, o **DFA**, è definito dalla seguente quintupla:

- Q è l'insieme finito di stati.
- Σ è l'alfabeto dell'automa, sul quale sono costruite le stringhe processate dall'automa.
- $f: Q \times \Sigma \rightarrow Q$ è la funzione di transizione, che indica in quale stato spostarsi quando si è in un certo stato q e si legge certo un simbolo dell'alfabeto.
- $q_1 \in Q$ è lo stato iniziale.
- $F \subseteq Q$ è il sottoinsieme degli stati finali, o stati accetta. Quando la computazione termina in uno stato $q \in F$, allora la stringa in input è accettata.

Una **Macchina di Turing** è una macchina a stati finiti costituita da un nastro infinito diviso in celle, su cui una testina è libera di muoversi. Ciascuna cella può contenere un simbolo ed inizialmente troviamo nelle prime k celle l'input, mentre in tutte le altre, da $k+1$ a infinito, un carattere blank, indicato con $\sqcup$. Gli stati accept e reject fanno terminare immediatamente la computazione una volta che vi si giunge.

Una configurazione è un modo compatto di indicare la situazione in cui si trova la macchina in un dato momento e viene indicata nella forma uqv , dove q è la sottostringa letta finora, q è lo stato in cui ci si trova e v è la sottostringa dei caratteri da leggere, con la testina che si trova sul primo carattere di v .

Variante Stayer della MdT:

consente alla testina di restare ferma, dunque la funzione di transizione presenterà $\{L, S, R\}$ al posto di $\{L, R\}$.

Una Stayer può simulare una MdT convenzionale semplicemente non usando l'opzione S . Una MdT convenzionale può simulare una Stayer modificando la sua funzione di transizione in questo modo: $\delta(q, a) = (q', a', S)$ viene scomposta in questi due passaggi:

- $\delta(q, a) = (qx, a', R)$, con qx stato aggiuntivo da usare come intermedio.
- $\delta(qx, x) = (q', x, L)$, per ogni simbolo x nell'alfabeto Γ , in modo da riportare la testina al suo posto originale.

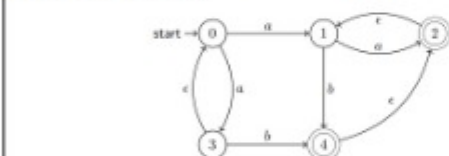
Variante multinastro della MdT:

è una normale MdT ma con $k \geq 1$ nastri, in cui l'input compare sul primo e gli altri sono inizialmente vuoti; k testine si muovono in modo indipendente tra loro. La funzione di transizione diventa $\delta: Q \times \Gamma^k \rightarrow Q \times \Gamma^k \times \{L, R\}^k$, che specifica i k simboli letti, i k simboli da scrivere ed i k movimenti delle testine. Una MdT multinastro può simulare una MdT semplice limitandosi ad usare il primo nastro. L'idea per simulare una MdT multinastro con una semplice è quella di disporre il contenuto dei k nastri in blocchi di lunghezza variabile, separati da un nuovo simbolo speciale: $\#$. Inoltre, per indicare la posizione della testina su ciascuno dei nastri, dovremo includere una x con un pallino sopra per ciascuna $x \in \Gamma$ (dunque blank incluso). Also, nessun nome particolare da usare per i caratteri con il pallino, è solo che non posso scriverli come testo, però sono esattamente delle lettere con un pallino annerito sopra.

La spiegazione dei passi salienti è la seguente:

- Per ogni istruzione della MdT multinastro M , la MdT mononastro S opera come segue:
1. Scorre i k blocchi e 'raccolge' informazioni sui k simboli letti, che corrispondono ai k simboli che la multinastro leggerebbe.
 2. Riporta la testina all'inizio del nastro.
 3. Simula la transizione di M , cioè scorre di nuovo i k blocchi e applica su ciascuno scrittura e spostamento come indica dalla funzione di transizione di M . (Praticamente, verrà sovrascritto ciascun simbolo con il pallino con il nuovo simbolo, ci si sposterà a dx o sx e si sovrascriverà il simbolo x trovato con la sua variante con il pallino)
 4. Entra nello stato che 'ricorda' il nuovo stato di M e riposiziona la testina all'inizio del nastro.
- S userà un numero molto grande di stati e mosse rispetto ad M , ma avrà lo stesso potere computazionale.

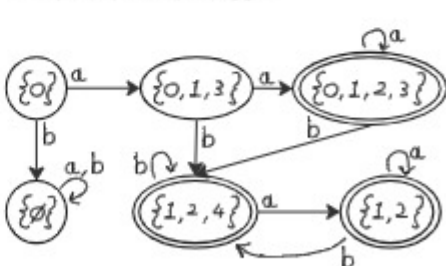
Trasformare il seguente NFA nel DFA equivalente (usando la tecnica del teorema di equivalenza tra DFA e NFA).



- $\{0\}a = \{0, 1, 3\}$ | $\{0, 1, 3\}a = \{0, 1, 2, 3\}$
 $\{0\}b = \{0\}$ | $\{0, 1, 3\}b = \{1, 2, 4\}$
 $\{0\}$ loop in se stesso $\forall x \in \Sigma$ | ...

- Q sarà uguale a tutti questi sottoinsiemi, quindi $\{\{0\}, \{0, 1, 3\}, \dots\}$
- L'alfabeto Σ sarà uguale a quello originale meno la ϵ
- $q_0 = \{0\}$
- La funzione di transizione è sempre $Q \times \Sigma$ e la tabella che ne risulta è riempita con gli insiemi trovati al passo iniziale.
- L'insieme degli stati finali F conterrà tutti gli insiemi di stati in cui compare almeno uno degli stati accettanti dell'automa originale, dunque in questo tutti quelli con 2 e 4.

L'idea di base è quella di trovare tutte le possibili combinazioni di insiemi di stati che possono verificarsi durante una computazione su questo NFA ed usarle per costruire un DFA equivalente. Si parte dallo stato iniziale e si controllano tutte le lettere dell'alfabeto: per ciascuna nuova combinazione di stati ottenuta, si ripete questo passaggio.



Un **NFA**, o **Automa Finito Non Deterministico**, è descritto dalla stessa quintupla, ma ha un comportamento leggermente diverso:

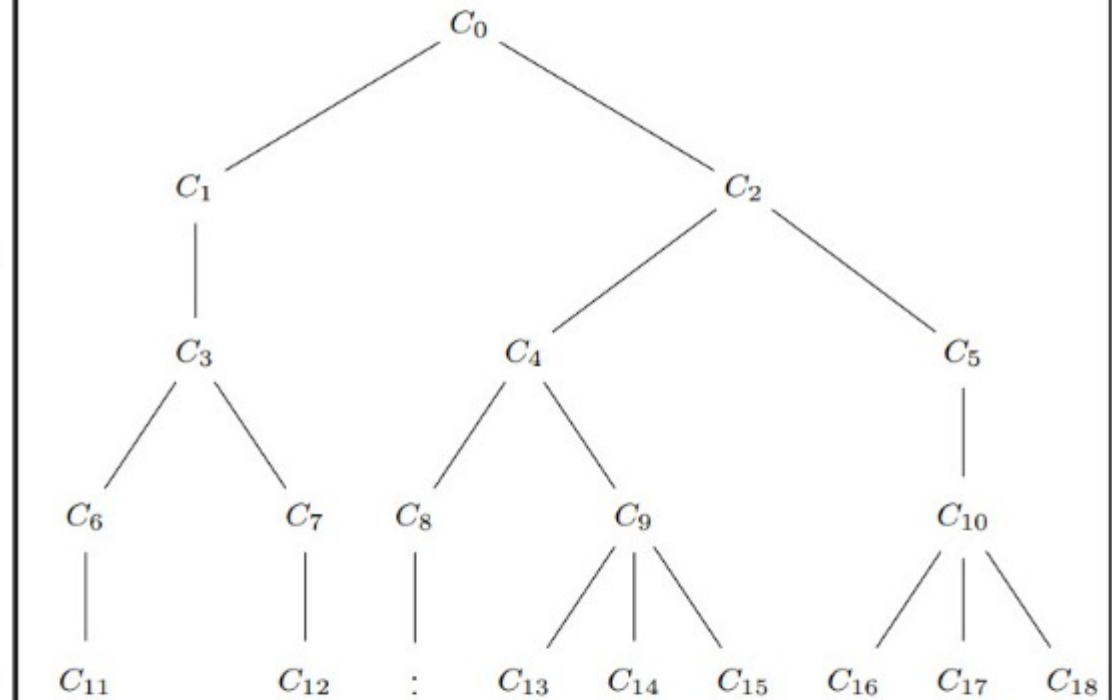
- La funzione di transizione restituisce un $P(Q)$ quando scriviamo la definizione della quintupla. Nel pratico si tratta di un insieme di stati, ad esempio $\{q_0\}, \{q_0, q_1\}$.
- Sono ammesse le epsilon-transitions, cioè delle transizioni 'speciali' dal seguente comportamento: se giungiamo in uno stato q' e da questo esce una freccia con una ϵ sopra verso q'' , allora in questo stesso passo ci stiamo spostando anche in q'' .
- Per uno stato q è possibile avere 0,1 o più archi uscenti per ciascun simbolo dell'alfabeto.
- Quando ci sono più scelte per un certo simbolo, la macchina si divide in più copie di se stessa e la computazione continua in parallelo: se una copia termina su uno stato finale, allora accetta, altrimenti rifiuta.

Una **Macchina di Turing Deterministica**, o **MdT** o **TM**, è definita dalla seguente settupla:

- Q è l'insieme di stati.
- Σ è l'alfabeto di lavoro, in cui il carattere $\sqcup$ non compare.
- Γ è l'alfabeto del nastro, in cui è presente il carattere $\sqcup$.
- $\delta: Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R\}$ è la funzione di transizione.
- $q_0 \in Q$ è lo stato iniziale.
- **qaccept** è lo stato accettante.
- **qreject** è lo stato che rifiuta.

Variante non deterministica della MdT:

si comporta allo stesso modo di un NFA e la funzione di transizione diventa $\delta: Q \times \Gamma = P(Q \times \Gamma \times \{L, R\})$, con la P che indica che lo spostarsi è una probabilità. La sua computazione genera un albero in cui i nodi sono le configurazioni raggiungibili a partire da quella iniziale, indicata con C_0 . L'idea per simularla con una MdT deterministica è tramite una visita BFS dell'albero, simulando quindi il primo passo di ogni computazione, poi il secondo di ognuna e così via, fermandosi non appena una di esse giunge in uno stato accettante. La visita BFS consente anche di evitare di entrare in loop.



- Nella simulazione da MdT non deterministica a MdT deterministica a 3 nastri, iniziamo definendo il ruolo di ciascun nastro:
- Il primo contiene sempre la stringa di input e non viene mai modificato.
 - Il secondo contiene una copia del contenuto del nastro di N corrispondente ad un ramo della sua computazione.
 - Il terzo serve a tener traccia della posizione di D nell'albero della computazione non deterministica di N .

Ogni nodo dell'albero può avere al massimo b figli, con $b = \#$ massimo di scelte di N . Ad ogni nodo assegniamo quindi un indirizzo, cioè una stringa sull'alfabeto $\{1, 2, \dots, b\}$. Se il nastro 3 contiene la stringa 212, allora D sta simulando la configurazione C_9 di N (i numeri indicano quale transizione scegliere, cioè quale figlio viene scelto, contando da sx a dx : secondo, poi primo, poi secondo). Poiché visitiamo in BFS, la configurazione successiva è C_{10} , rappresentata dalla stringa 221.

Sul nastro 3 vengono generate **tutte** le stringhe corrispondenti ai numeri crescenti dell'alfabeto b -ario, quindi: $1, 2, \dots, b, 11, 12, \dots, 1b, \dots$ NB: le stringhe possibili compaiono tutte, ma quelle che non corrispondono a delle computazioni di N sono ignorate.

I passi con cui sintetizzare la simulazione sono i seguenti:

1. Scrivi 1 sul nastro 3.
2. Copia l'input w del nastro 1 sul nastro 2.
3. Se il nastro 3 contiene la codifica di un indirizzo di una computazione di N , allora esegui la computazione usando il nastro 2 per simulare N con input w .
4. Se sul nastro 2 si raggiunge una configurazione accettante, allora accetta.
5. Incrementa di 1 il numero b -ario sul nastro 3.
6. Vai al passo 2.